```
SEPARATE(¬IDL )¶--|----------------------------------------------------------------------------------------------------¶--|¬¬INIT_GRMR¶--|-------------
---------------------------------------------------------------------------------¶PROCEDURE¬INIT_GRMR¬( NOM_TEXTE :STRING¬) IS¶  USE  TERM_LIST;¶¶
 GRAMMAR¬¬: TREE;¶  GR_RULE_SEQ¬: SEQ_TYPE;¶¶  MORE_PASSES¬: BOOLEAN;¬¬¬¬¬¬¬¬--| INDIQUE DES CHANGEMENTS DANS LA FERMETURE TRANSITIVE¶  PASS¬¬: INTEGER
¬¬:= 0;¶¶  --|-------------------------------------------------------------------------------------------------¶  --|¬PROCEDURE¬INITIALIZE¶  PROCEDURE I
NITIALIZE IS¶    RULE_SEQ¬¬: SEQ_TYPE¬:= GR_RULE_SEQ;¶    RULE¬¬¬: TREE;¶    RULE_INIT_LIST¬¬: SEQ_TYPE;¶    RULEINFO¬¬: TREE;¶    ALT_SEQ¬¬: SEQ_TYPE
;¶    ALT¬¬¬: TREE;¶    SYL_SEQ¬¬: SEQ_TYPE;¶    SYL¬¬¬: TREE;¶    IS_NULLABLE¬¬: BOOLEAN;¬¬¬¬¬¬¬-- CURRENT RULE HAS¬NULLABLE ALT¶    GENS_TER_STR¬¬: B
OOLEAN;¬¬¬¬¬¬¬-- CURRENT RULE HAS¬TERMINAL ALT¶    NONTER_NAME¬¬: TREE;¬¬¬¬¬¬¬¬-- SYMBOL_REP OF NON-TERMINAL¶    NONTER_DEF_LIST¬¬: SEQ_TYPE;¶    RULE_C
OUNT¬¬: INTEGER¬:= 0;¶    INIT_NONTER_S¬¬: TREE;¶  BEGIN¶    WHILE¬NOT IS_EMPTY( RULE_SEQ ) LOOP¬¬¬¬¬¬¬¬--| TANT QU'IL Y A DES RGLES¶      POP( RULE_SE
Q, RULE );¬¬¬¬¬¬¬¬¬--| EN EXTRAIRE UNE¶      RULE_COUNT :=¬RULE_COUNT + 1;¬¬¬¬¬¬¬¬--| UNE DE PLUS VUE¶¶      RULE_INIT_LIST := (TREE_NIL, TREE_NIL);¶
   RULEINFO¬:= MAKE( DN_RULEINFO );¬¬¬¬¬¬¬--| FABRIQUER SON INFO¶      INIT_NONTER_S¬:= MAKE( DN_RULE_S );¬¬¬¬¬¬¬--| FABRIQUER UNE LISTE DE RGLES¶
LIST( INIT_NONTER_S, INSERT ( (TREE_NIL, TREE_NIL), RULE ) );¬¬¬¬¬--| Y METTRE LA RGLE¶      D  ( XD_INIT_NONTER_S, RULEINFO, INIT_NONTER_S );¬¬¬¬¬¬--|
PORTER CELA DANS L'INFO¶      DB ( XD_IS_REACHABLE,  RULEINFO, FALSE );¬¬¬¬¬¬¬--| METTRE REACHABLE  FAUX¶      DI ( XD_RULE_NBR,      RULEINFO, RULE_CO
UNT¬);¬¬¬¬¬¬--| Y PORTER LE N DE RGLE¶      DI ( XD_TIMECHANGED,   RULEINFO, 0 );¬¬¬¬¬¬¬--| METTRE  0 TIMECHANGED¶      DI ( XD_TIMECHECKED,   RULEINFO,
 0 );¬¬¬¬¬¬¬--| ET TIMECHECKED¶      D  ( XD_RULEINFO, RULE,¬RULEINFO );¬¬¬¬¬¬¬--| POINTER L'INFO DANS LA RGLE¶      IS_NULLABLE := FALSE;¶      GENS_TE
R_STR := FALSE;¶¶      ALT_SEQ := LIST( RULE );¬¬¬¬¬¬¬¬--| PRENDRE LA LISTE D'ALTERNATIVES¶      WHILE NOT IS_EMPTY( ALT_SEQ ) LOOP¬¬¬¬¬¬¬--| TANT QU'IL
 Y A DES ALTERNATIVES¶        POP( ALT_SEQ, ALT );¬¬¬¬¬¬¬¬¬--| EN EXTRAIRE UNE¶        D¬ ( XD_RULE, ALT, RULE );¬¬¬¬¬¬¬¬--| MENTIONNER LA RGLE QUI LA C
ONTIENT¶        DECLARE¶¬ALT_NOT_NULLABLE¬: BOOLEAN¬:= FALSE;¶¬ALT_NOT_GEN_TER_STR¬: BOOLEAN¬:= FALSE;¶        BEGIN¶¬SYL_SEQ := LIST( ALT );¬¬¬¬¬¬¬¬--
| PRENDRE LA LISTE DE SYLLABES DE L'ALTERNATIVE¶¬IF NOT IS_EMPTY( SYL_SEQ ) THEN¬¬¬¬¬¬¬--| S'IL Y A DES SYLLABES¶¬  SYL := HEAD( SYL_SEQ );¬¬¬¬¬¬¬¬--| P
RENDRE LA TTE¶¬  IF SYL.TY = DN_TERMINAL THEN¬¬¬¬¬¬¬--| SI C'EST UN TERMINAL¶¬    RULE_INIT_LIST := UNION( RULE_INIT_LIST, SYL );¬¬¬¬¬--| L'AJOUTER  LA
LISTE DES TTES TERMINALES D'ALTERNATIVES¶¬  END IF;¶¬  WHILE NOT IS_EMPTY( SYL_SEQ¬) LOOP¬¬¬¬¬¬¬--| TANT QU'IL Y A DES SYLLABES¶¬    POP( SYL_SEQ, SYL
);¬¬¬¬¬¬¬¬--| EN EXTRAIRE UNE¶¬    IF SYL.TY = DN_TERMINAL THEN¬¬¬¬¬¬¬--| SI C'EST UN TERMINAL¶¬      ALT_NOT_NULLABLE := TRUE;¬¬¬¬¬¬¬--| INDIQUER NON AN
NULABLE¶¬    ELSE¬¬¬¬¬¬¬¬¬¬--| C'EST¬UN NON TERMINAL¶¬      NONTER_NAME := D( XD_SYMREP, SYL );¶¬      NONTER_DEF_LIST := LIST( NONTER_NAME );¶¬      W
HILE NOT IS_EMPTY( NONTER_DEF_LIST ) AND THEN HEAD( NONTER_DEF_LIST).TY /= DN_RULE¬LOOP¶¬        NONTER_DEF_LIST := TAIL( NONTER_DEF_LIST );¶¬      EN
D¬LOOP;¶¬      IF IS_EMPTY( NONTER_DEF_LIST ) THEN¶¬        ERROR( D( LX_SRCPOS, SYL ), "NON-TERMINAL NOT DEFINED - " & PRINT_NAME(¬NONTER_NAME ) );¶¬
        D( XD_RULE,¬SYL, TREE_VOID );¶¬        ALT_NOT_NULLABLE := TRUE;¶¬      ELSE¶¬        D( XD_RULE,¬SYL, HEAD( NONTER_DEF_LIST ) );¬¬¬¬¬-- ASSUME
¬THE WORST¬ABOUT THE¬NON-TERMINAL FOR NOW¶¬        ALT_NOT_NULLABLE    := TRUE;¶¬        ALT_NOT_GEN_TER_STR := TRUE;¶¬      END¬IF;¶¬    END IF;¶¬  E
ND LOOP;¶¬END IF;¶¬IF NOT ALT_NOT_NULLABLE THEN¶¬  IS_NULLABLE := TRUE;¶¬END IF;¶¬IF NOT ALT_NOT_GEN_TER_STR THEN¶¬  GENS_TER_STR := TRUE;¶¬END IF;¶
      END;¶      END¬LOOP;¶¶      LIST( RULEINFO,RULE_INIT_LIST );¶      DB¬( XD_GENS_TER_STR, RULEINFO, GENS_TER_STR );¶      DB¬( XD_IS_NULLABLE, RU
LE, IS_NULLABLE );¶    END LOOP;¶¬      -- FIRST RULE¬IS ALWAYS¬REACHABLE¶    DB( XD_IS_REACHABLE, D( XD_RULEINFO, HEAD( GR_RULE_SEQ ) ), TRUE );¶  EN
D INITIALIZE;¶  --|-----------------------------------------------------------------------------------------------¶  --|¬PROCEDURE¬TRANS_CLOSE¶  PROCE
DURE TRANS_CLOSE IS¶    RULE_SEQ¬¬: SEQ_TYPE¬:= GR_RULE_SEQ;¶    RULE¬¬¬: TREE;¶    RULE_INIT_LIST¬¬: SEQ_TYPE;¶    RULEINFO¬¬: TREE;¶    ALT_SEQ¬¬: S
EQ_TYPE;¶    ALT¬¬¬: TREE;¶    SYL_SEQ¬¬: SEQ_TYPE;¶    SYL¬¬¬: TREE;¶    IS_NULLABLE¬¬: BOOLEAN;¬¬¬¬¬¬¬-- CURRENT RULE HAS¬NULLABLE ALT¶    IS_REACHAB
LE¬¬: BOOLEAN;¬¬¬¬¬¬¬-- CURRENT RULE IS REACHABLE¶    GENS_TER_STR¬¬: BOOLEAN;¬¬¬¬¬¬¬-- CURRENT RULE HAS¬TERMINAL ALT¶    ALT_NOT_NULLABLE¬: BOOLEAN;¬¬¬
¬¬¬-- ALT FOUND TO BE NOT NULLABLE¶    ALT_NOT_GEN_TER_STR¬: BOOLEAN;¶¬      -- ALT FOUND TO BE NOT TERMINALN¶    NONTER_RULE¬¬: TREE;¶    NONTER_INFO
¬¬: TREE;¶    TIMECHANGED¬¬: INTEGER;¶    TIMECHECKED¬¬: INTEGER;¶    NONTER_CHANGED¬¬: INTEGER;¶    CHANGE_FLAG¬¬: BOOLEAN;¬¬¬¬¬¬¬-- RULE CHANGED IN T
HIS PASS¶    INIT_NONTER_S¬¬: TREE;¶    INIT_NONTER_SEQ¬¬: SEQ_TYPE;¶¶  BEGIN¶    WHILE¬NOT IS_EMPTY( RULE_SEQ ) LOOP¶      RULE      := HEAD( RULE_SEQ
 );¶      RULE_SEQ := TAIL( RULE_SEQ );¶¶      RULEINFO := D( XD_RULEINFO, RULE );¶      TIMECHANGED := DI( XD_TIMECHANGED, RULEINFO¬);¶      TIMECHEC
KED := DI( XD_TIMECHECKED, RULEINFO¬);¶      DI(¬XD_TIMECHECKED, RULEINFO, PASS );¶¶      RULE_INIT_LIST := LIST(¬RULEINFO );¶      IS_REACHABLE¬ := D
B( XD_IS_REACHABLE, RULEINFO );¶      IS_NULLABLE¬ := FALSE;¬¬¬¬¬¬¬¬-- WE'LL SEE IF IT CHANGES¶      GENS_TER_STR¬ := DB( XD_GENS_TER_STR, RULEINFO );¶
      INIT_NONTER_S¬ := D( XD_INIT_NONTER_S, RULEINFO );¶      INIT_NONTER_SEQ:= LIST(¬INIT_NONTER_S );¶      CHANGE_FLAG := FALSE;¶¶      ALT_SEQ :=
LIST( RULE );¶      WHILE NOT IS_EMPTY( ALT_SEQ ) LOOP¶        POP( ALT_SEQ, ALT );¶¶        ALT_NOT_NULLABLE   :=¬FALSE;¶        ALT_NOT_GEN_TER_STR:
=¬FALSE;¶        SYL_SEQ¬        :=¬LIST ( ALT);¶        IF NOT IS_EMPTY( SYL_SEQ ) THEN¶¬SYL := HEAD( SYL_SEQ );¶¬IF SYL.TY¬= DN_NONTERMINAL THEN¶¬  N
ONTER_RULE := D(¬XD_RULE, SYL );¶¬  IF NONTER_RULE.TY¬/= DN_VOID THEN¶¬    INIT_NONTER_SEQ¬:= R_UNION( INIT_NONTER_SEQ, LIST( D( XD_INIT_NONTER_S, D(
XD_RULEINFO, NONTER_RULE ) ) )¬);¶¬  END IF;¶¬END IF;¶        END IF;¶        WHILE NOT IS_EMPTY( SYL_SEQ ) LOOP¶¬POP( SYL_SEQ, SYL );¶¬IF SYL.TY¬= DN
_TERMINAL THEN¶¬  IF NOT ALT_NOT_NULLABLE THEN¶¬    ALT_NOT_NULLABLE := TRUE;¶¬    IF TIMECHANGED >= TIMECHECKED THEN¶¬¬¬¬¬¬¬-- OTHERWISE ALREADY DONE¶
¬      RULE_INIT_LIST := UNION( RULE_INIT_LIST, SYL );¶¬    END IF;¶¬  END IF;¶¬ELSE -- SINCE IT'S DN_NONTERMINAL¶¬  NONTER_RULE := D(¬XD_RULE, SYL );
¶¬  IF NONTER_RULE.TY¬= DN_VOID¬THEN¶¬    ALT_NOT_NULLABLE := TRUE;¶¬  ELSE¶¬    NONTER_INFO := D( XD_RULEINFO, NONTER_RULE );¶¬    NONTER_CHANGED :=
DI( XD_TIMECHANGED, NONTER_INFO );¶¬    IF TIMECHANGED >= TIMECHECKED AND THEN IS_REACHABLE AND¬THEN NOT DB( XD_IS_REACHABLE,¬NONTER_INFO ) THEN¶¬
  MORE_PASSES := TRUE;¶¬      DB(¬XD_IS_REACHABLE, NONTER_INFO,¬TRUE);¶¬      DI(¬XD_TIMECHANGED, NONTER_INFO, PASS );¶¬    END IF;¶¬    IF NOT ALT_NO
T_NULLABLE THEN¶¬      IF NOT DB( XD_IS_NULLABLE, NONTER_RULE ) THEN¶¬        ALT_NOT_NULLABLE := TRUE;¶¬      ELSE¶¬        IF NONTER_CHANGED > TIMEC

HANGED¬THEN¶¬¬¬¬¬¬¬¬¬-- KEEP LOOKING IF NONTER BECAME NULLABLE¶¬¬TIMECHANGED := NONTER_CHANGED;¶¬        END IF;¶¬      END¬IF;¶¬      IF NONTER_CHANGE
D >= TIMECHECKED THEN¶¬        RULE_INIT_LIST := UNION ( RULE_INIT_LIST,¬LIST ( NONTER_INFO ) );¶¬      END¬IF;¶¬    END IF;¶¬    IF NOT ALT_NOT_GEN_T
ER_STR AND THEN¬NOT GENS_TER_STR AND THEN NOT¬DB ( XD_GENS_TER_STR, NONTER_INFO ) THEN¶¬      ALT_NOT_GEN_TER_STR := TRUE;¶¬    END IF;¶¬  END IF;¶¬EN
D IF;¶¬EXIT WHEN¬ALT_NOT_NULLABLE AND THEN GENS_TER_STR AND THEN TIMECHANGED¬< TIMECHECKED;¶        END LOOP;¶        IF NOT ALT_NOT_NULLABLE THEN¶¬IS
_NULLABLE := TRUE;¶        END IF;¶        IF NOT ALT_NOT_GEN_TER_STR THEN¶¬GENS_TER_STR := TRUE;¶        END IF;¶      END¬LOOP;¶¶      IF NOT SAME (
¬LIST ( RULEINFO ), RULE_INIT_LIST ) THEN¶        LIST ( RULEINFO, RULE_INIT_LIST¬);¶        CHANGE_FLAG¬:= TRUE;¶      END¬IF;¶      IF NOT SAME (¬LI
ST ( INIT_NONTER_S), INIT_NONTER_SEQ ) THEN¶        LIST ( INIT_NONTER_S,¬INIT_NONTER_SEQ );¶        CHANGE_FLAG¬:= TRUE;¶      END¬IF;¶      IF GENS_
TER_STR AND THEN NOT DB (¬XD_GENS_TER_STR, RULEINFO ) THEN¶        DB ( XD_GENS_TER_STR,¬RULEINFO,¬TRUE );¶        CHANGE_FLAG¬:= TRUE;¶      END¬IF;¶
      IF IS_NULLABLE AND THEN¬NOT DB ( XD_IS_NULLABLE, RULE¬) THEN¶        DB ( XD_IS_NULLABLE, RULE, TRUE¬);¶        CHANGE_FLAG¬:= TRUE;¶      END¬I
F;¶      IF CHANGE_FLAG THEN¶        DI ( XD_TIMECHANGED, RULEINFO, PASS );¶        MORE_PASSES¬:= TRUE;¶      END¬IF;¶    END LOOP;¶  END TRANS_CLOSE
;¶  --|-------------------------------------------------------------------------------------------------¶  --|¬PROCEDURE¬CHECK_GRAMMAR¶  PROCEDURE CHECK
_GRAMMAR IS¶    RULE_SEQ¬: SEQ_TYPE¬:= GR_RULE_SEQ;¶    RULE¬¬: TREE;¶    RULEINFO¬: TREE;¶  BEGIN¶    WHILE¬NOT IS_EMPTY( RULE_SEQ ) LOOP¶      POP(
RULE_SEQ, RULE );¶      RULEINFO := D( XD_RULEINFO, RULE );¶      IF NOT DB( XD_IS_REACHABLE, RULEINFO ) THEN¶        ERROR( D( LX_SRCPOS,RULE ), "RUL
E CANNOT BE REACHED - " & PRINT_NAME( D( XD_NAME,¬RULE ) ) );¶      END¬IF;¶      IF NOT DB( XD_GENS_TER_STR, RULEINFO ) THEN¶        ERROR( D( LX_SRC
POS,RULE ), "DOES NOT GEN TERMINAL STRING - "¬& PRINT_NAME( D( XD_NAME, RULE ) ) );¶      END¬IF;¶    END LOOP;¶  END CHECK_GRAMMAR;¶¶BEGIN¶¶  PUT_LIN
E( "INITIALIZE.");¶  DECLARE¶    USER_ROOT¬: TREE;¶  BEGIN¶    OPEN_IDL_TREE_FILE( NOM_TEXTE & ".lar" );¶    USER_ROOT := D(¬XD_USER_ROOT, TREE_ROOT )
;¶    GRAMMAR   := D(¬XD_GRAMMAR, USER_ROOT );¶  END;¶¶  GR_RULE_SEQ := LIST( GRAMMAR );¶  INITIALIZE;¶¶  LOOP¶    PASS := PASS + 1;¶    PUT (¬"BEGIN
TRANS CLOSE PASS " );¶    INT_IO.PUT ( PASS, 1 );¶    PUT_LINE ( "." );¶    MORE_PASSES := FALSE;¶    TRANS_CLOSE;¶    EXIT WHEN NOT MORE_PASSES;¶  EN
D LOOP;¶¶  PUT_LINE ( "CHECK GRAMMAR."¬);¶  CHECK_GRAMMAR;¶¶  CLOSE_IDL_TREE_FILE;¶--|----------------------------------------------------------------
----------------------------------¶END INIT_GRMR;¶
```